Distributed Systems

12th Lecture

Eom, Hyeonsang (엄현상)
Department of Computer Science & Engineering
Seoul National University

©Copyrights 2026 Eom, Hyeonsang All Rights Reserved

Outline

- Architectures
- Peer-to-Peer Computing
  - Introduction
  - Chord
- Q&A

Architectures

- Software Architecture
  - How Software Components Are Organized
  - How Software Components Should Interact
- System Architecture
  - Final Instantiation of a Software Architecture
- Important Styles of Architecture for (Autonomic) Distributed Systems
  - Layered Architectures
  - Object-Based Architectures
  - Data-Centered Architectures
  - Event-Based Architectures

Architectural Styles

Figure 2-1 on Page 35

(a) layered architectural style

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectural Styles (Cont'd)

Figure 2-1 on Page 35

(b) The object-based architectural style

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectural Styles (Cont'd)

Figure 2-2 on Page 36

(a) The event-based architectural style

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Architectural Styles (Cont'd)

Figure 2-2 on Page 36

(b) The shared data-space architectural style

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

System Architectures

- Centralized Architecture
  - Clients That Request Services from Servers
  - Support for Vertical Distribution
    - Placing different components on different machines
- Decentralized Architecture
  - Process Being a Client and a Server
  - Support for Horizontal Distribution
    - Spitting up a client or server physically into logically equivalent parts with each part operating on its own share of data set

Peer-to-Peer Architectures

- Overlay Network
  - Network in which the nodes are formed by the processes and the links represent the possible communication channels
- Structured P2P Architecture
  - Overlay network is Constructed using a deterministic procedure
    - Distributed Hash Table (DHT)
- Unstructured P2P Architecture
  - Overlay network is Constructed using a random algorithm

Centralized Architectures

Figure 2-3 on Page 37

General interaction between a client and a server

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Application Layering

- Following Layered Architectural Style
  - User-Interface Level
  - Processing Level
  - Data level

Application Layering (Cont'd)

Figure 2-4 on Page 39

The simplified organization of an Internet search engine into three different layers

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Multitiered Architectures

- The simplest organization is to have only two types of machines:
- A client machine containing only the programs implementing (part of) the user-interface level
- A server machine containing the rest,
  - the programs implementing the processing and data level

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Multitiered Architectures (Cont'd)

Figure 2-5 on Page 41

Alternative client-server organizations (a)–(e)

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Multitiered Architectures (Cont'd)

Figure 2-6 on Page 43

An example of a server acting as client

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Structured Peer-to-Peer Architectures

Figure 2-7 on Page 45

The mapping of data items onto nodes in Chord

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Structured Peer-to-Peer Architectures (Cont'd)

Figure 2-8 on Page 46

The mapping of data items onto nodes in CAN

Tanenbaum & Van Steen, Distributed Systems: Principles and Paradigms, 2e, (c) 2007 Prentice-Hall, Inc. All rights reserved

Peer-to-Peer (P2P) Computing

- Definition
  - Computing by Sharing Data & Resources on a Very Large Scale w/o Server Requirements
- Important Characteristics
  - Each Node's Resource Contribution
  - Same Functional Capabilities & Responsibilities of Nodes
  - No Central Administration
  - Limited Degree of Anonymity
  - Unpredictable Availability
  - Fault Tolerance

Key Issue: Efficient Data Placement & Access

1st-Generation P2P Systems

- File Sharing and Storage Applications
  - Napster Music Exchange Service
    - Use of central servers to locate files
  - Gnutella
    - Distributed service using scoped broadcast queries

Main Problem: Limited Scalability or No Guarantee That Files Can Be Located

2nd-Generation P2P Systems

- Middleware
  - Application-Independent Management of Distributed Resources on a Global Scale
    - Routing Overlay for locating nodes and objects
      - Scalable
      - Load balanced
      - Adaptive to network dynamics
      - Fault tolerant
      - Efficiently discovering
      - Secure

Using Randomly Distributed Keys to Determine the Placement of Objects and to Retrieve Them

Implementing Key-Based Routing (KBR) Interface: Routing of Messages to a Live Node Responsible for the Destination Key

IP vs Overlay Routing

Figure 10.1 on Page 400

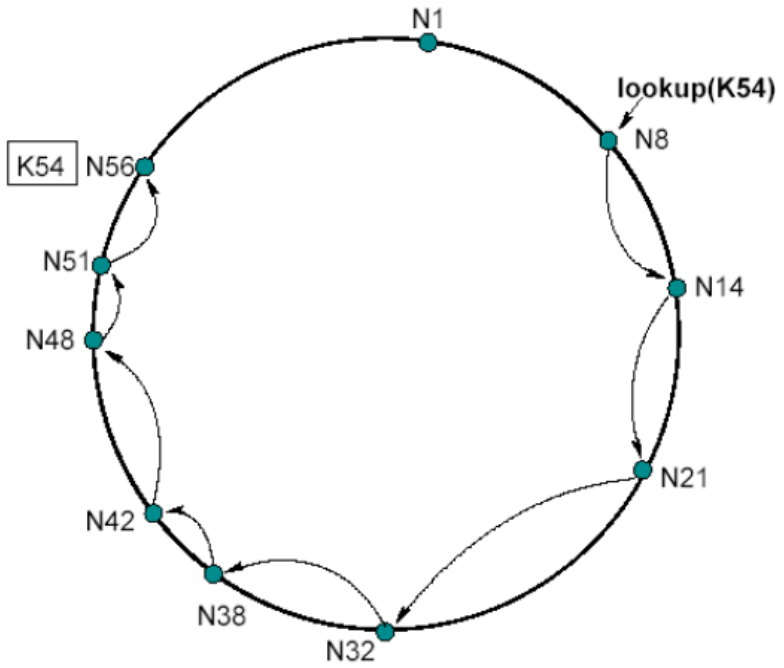
Structured P2P Overlay Networks

- Supporting Higher-Level Interfaces
  - Distributed Hash Table (DHT)
    - Basic Interface: put(), get(), remove()
    - E.g., Pastry
  - Distributed Object Location & Routing (DOLR)
    - Basic Interface: publish(), unpublish(), routeToObject()
    - E.g., Tapestry
- Ignoring/Considering Network Distances
  - Shortest Overlay-Hop Routing
    - E.g., Chord
  - Locally Optimal Routing
    - E.g., Tapestry

Chord Protocol [Keifer03]

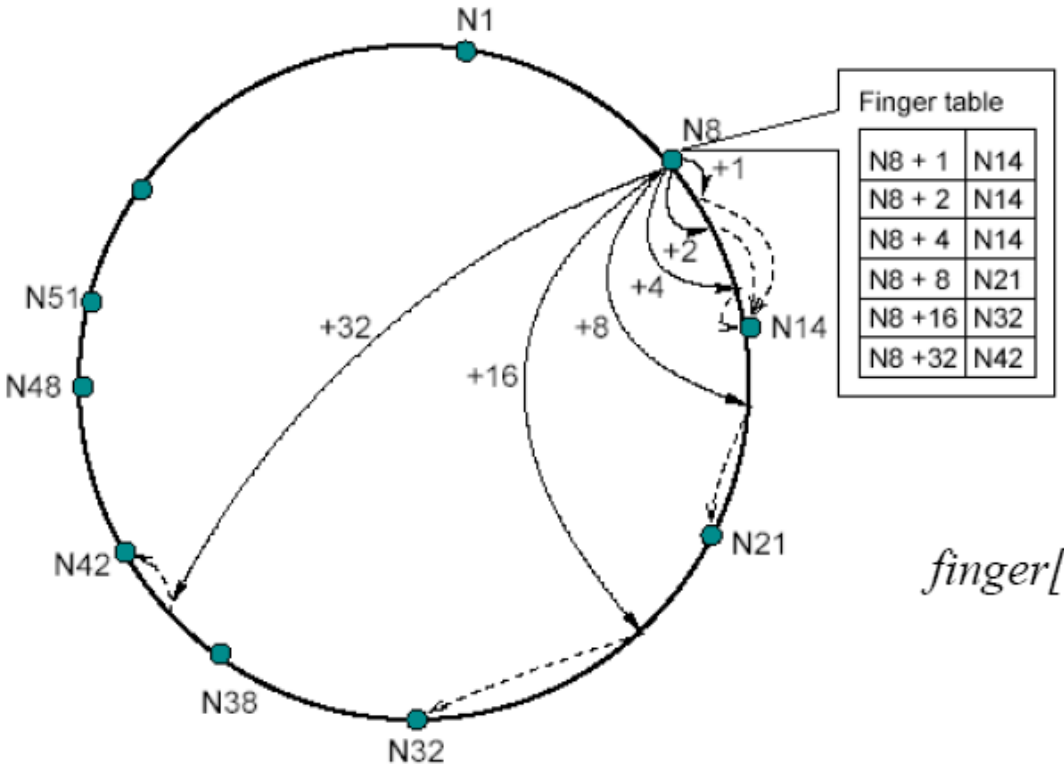
- Simple Key Location

```
// ask node n to find the successor of id
n.find_successor(id)
  if (id ϵ (n; successor])
    return successor;
  else
    // forward the query around the
      circle
   return successor.find_successor(id);
```



Cord Protocol (Cont'd)

- Scalable Key Location



$$finger[i] = successor\ (n + 2^{i-1})$$

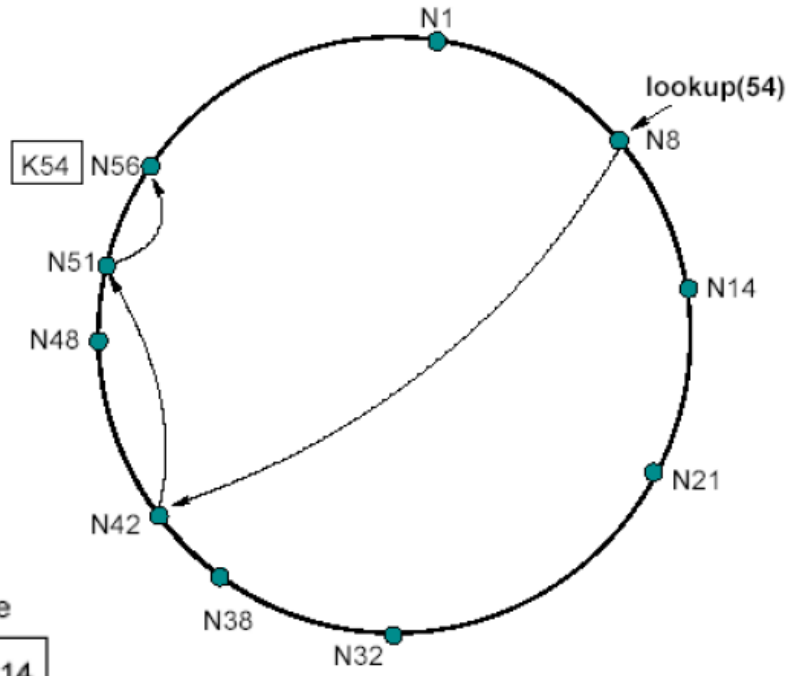
Cord Protocol (Cont'd)

- Scalable Key Location

```
// ask node n to find the successor of id
n.find_successor(id)
if (id ϵ (n; successor])
return successor;
else n0 = closest_preceding_node(id);
return n0.find successor(id);

// search the local table for the highest
// predecessor of id
n.closest_preceding_node(id)
for i = m downto 1
if (finger[i] ϵ (n; id))
return finger[i];
return n;
```

Is This Necessary?

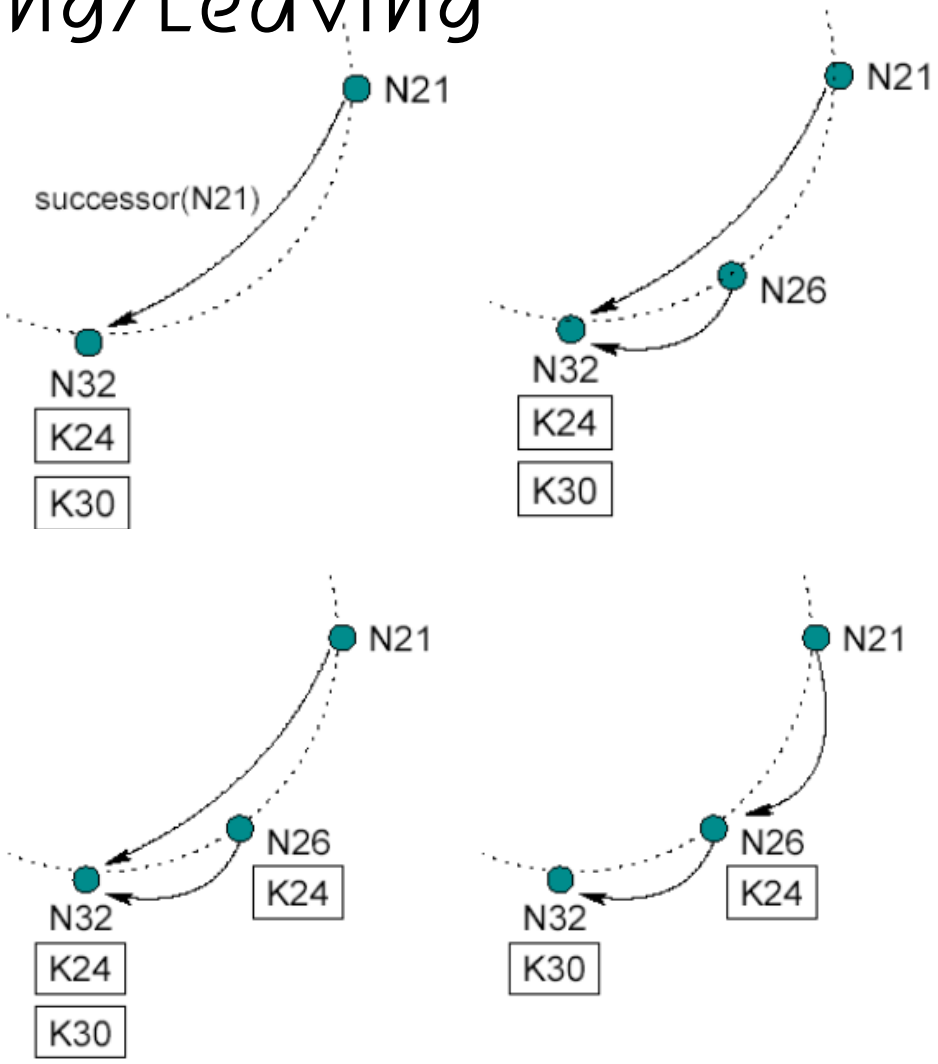


Finger table

N8 + 1	N14
N8 + 2	N14
N8 + 4	N14
N8 + 8	N21
N8 +16	N32
N8 +32	N42

Cord Protocol (Cont'd)

- Node Joining/Leaving



Cord Protocol (Cont'd)

- Properties of Chord
  - Load Balance
    - Acting as a Distributed Hash Function
  - Decentralization
    - Fully distributed
  - Scalability
    - Lookup cost growing as the log of # of nodes
  - Availability
    - Enabling the node responsible for a key to be found via automatic internal-table adjustment
  - Flexible naming
    - Using flat key-space

Reference

- [Keifer03] C. Keifer, "Cord: A Scalable Peer-to-Peer Look-Up Protocol for Internet Applications (by R. Morris, *et al*)," Writeup, Department of Computer Science, Saarland University, November 2002